UNIVERSITAT DE BARCELONA

Laboratori 4. Pràctica 2

Estructura de Dades

Grau en Enginyeria Informàtica

Facultat de Matemàtiques i Informàtica,

Universitat de Barcelona

Introducció

- **Objectiu**:
  - Treballar amb estructures lineals

- **Exercicis:**
  - **Exercici 1:** TAD PilaEstatica + main
  - **Exercici 2:** Ús TAD PilaEstatica
  - **Exercici 3:** TAD LinkedList

Planificació

- **Setmana 1 (del 2 de març al 8 de març) - LAB 4**
  - **EXAMEN PRÀCTICA 1**
  - Implementació de l'exercici 1
- **Setmana 2 (del 9 de març al 15 de març) - LAB 5**
  - Implementació de l'exercici 2
  - Implementació de l'exercici 3
- **Setmana 3 (del 16 de març al 20 de març) - LAB 6**
  - Implementació de l'exercici 3 i comentar el codi
  - Tasca al campus virtual per lliurar tota la pràctica

El lliurament final de la pràctica serà el **dia 22 de març de 2026** al campus virtual. Recordeu que s'han de lliurar tots els exercicis en el **fitxer .zip** al campus virtual.

Prova Pràctica 2

La Setmana del 23 al 27 de Març hi haurà la prova pràctica de la pràctica 2 al LAB7.

- És obligatori lliurar la pràctica al campus virtual, com a molt tard el dia 22 de març de 2026
- La prova consistirà en una modificació o ampliació de la pràctica lliurada

La nota d'aquesta pràctica és la nota de la prova pràctica

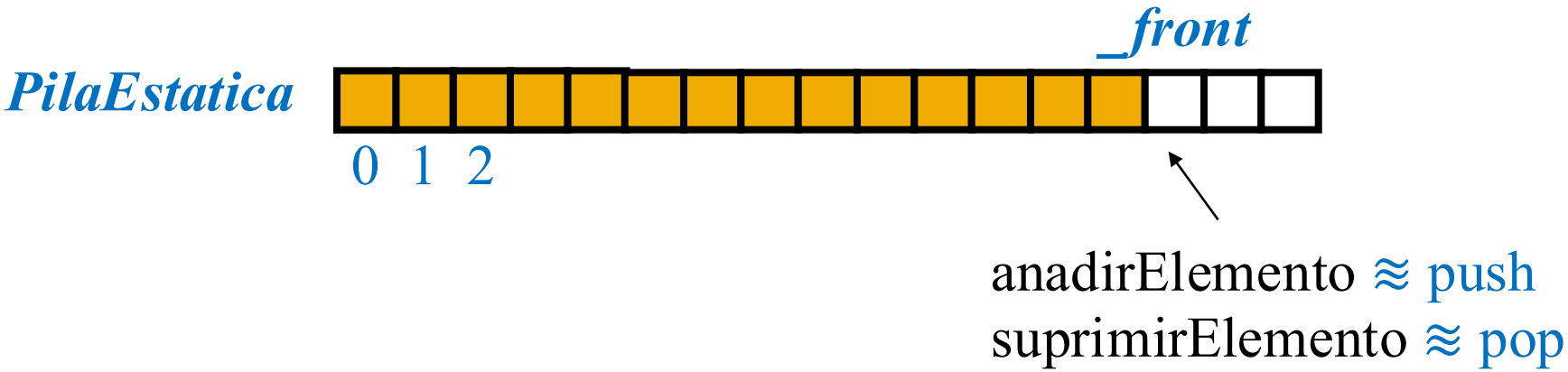
- La prova és presencial i s'ha de fer en el laboratori que cadascú tingui assignat

Exercici 1

- **Objectiu**
  - Definir i implementar el **TAD PilaEstatica**
- **Definir i implementar usant templates**
  - La representació del TAD es farà usant un vector
  - Les operacions són les definides a l'enunciat
  - S'han de controlar les excepcions
- Fer un **main.cpp** que usi el TAD PilaEstatica i que contingui els dos casos de prova i un menú
  - Feu una funció casProva1() que crideu al main
  - Feu una funció casProvaMenu() que crideu al main per fer el menú
  - Feu una funció casProvaEx2() que crideu al problema de l'exercici 2
  - Feu el main amb la crida a les tres funcions anteriors

Pila

- Un Stack o Pila és una estructura (Last-In First-Out)
- S'insereix al top de la pila i s'elimina des del top (a la practica es diu _encima) de la pila



PilaEstatica

☞ **NO podeu modificar el PilaEstatica.h, ni afegir més mètodes o atributs**

```
template <class Elemento> class PilaEstatica {
public:
  PilaEstatica(int max = TAMANO_MAXIMO);     // Constructor
  PilaEstatica(const PilaEstatica &original);// Constructor copia
  PilaEstatica( initializer_list<Elemento> elementos); //Inicializa la pila con una lista de elementos
  PilaEstatica(const vector<Elemento>& elementos); // Inicializa la pila a partir de un vector
  int tamano() const;      // Devuelve la longitud de la pila
  bool estaVacia() const;  // Devuelve true si la pila está vacía
  bool estaLlena() const;  // Devuelve true si la pila está llena
  const Elemento &elementoEncima() const; // Devuelve el elemento de encima, exception si está vacía
  void anadirElemento(const Elemento &e); // Añade un nuevo elemento, exception si no es posible
  void suprimirElemento(); // Elimina el elemento de encima de la pila,  exception si no es posible
  void print() const; // Imprime la pila completa por stdout
private:
  enum { TAMANO_MAXIMO = 30 };
  vector<Elemento> _datos;
  int _capacidad;
  int _encima; // Posición del elemento de encima (el top) de la pila
};
 #endif // PILA_ESTATICA_H
```

PilaEstatica

- La implementació es farà usant **vector**
- Sobre el codi …
  - Optimitzeu les operacions del TAD amb el **menor cost computacional** que sigui possible
  - Comenteu el codi (les operacions del TAD i el main)
  - Poseu noms entenedors a les variables i als mètodes
  - Identeu el codi adequadament

PilaEstatica - Main

1. Realitzeu la implementacio del TAD PilaEstatica
2. Tot seguit, codifiqueu un main.cpp amb el cas de prova 1, casProva1() i un altra funció, casProvaMenu() pel menú amb les següents opcions:

```
1. Inserir element a la pila
2. Suprimir element de la pila
3. Consultar el top de la pila
4. Imprimir tot el contingut de la PilaEstatica
5. Imprimir la posició del top de la pila
6. Sortir
```

PilaEstatica - Main

Cas de prova 1:

☞ **És necessari controlar excepcions al main**
Afegiu el cas de prova com el primer mètode del main

Pas	Entrada	Sortida
1	Crear una PilaEstatica de mida 3	Estructura creada
2	Inserir element 10	Element 10 agregat
3	Inserir element 11	Element 11 agregat
4	Imprimir front	Front: 11
5	Inserir element 12	Element 12 agregat
6	Inserir element 13	EXCEPTION: L'estructura està plena
7	Imprimir PilaEstatica	[10, 11, 12]
8	Imprimir front	Front: 12
9	Suprimir element	Element 12 eliminat
10	Inserir element 14	Element 14 agregat
11	Imprimir PilaEstatica	[10, 11, 14]
12	Imprimir front	Front: 14
13	Suprimir element	Element 14 eliminat
14	Suprimir element	Element 11 eliminat
15	Suprimir element	Element 10 eliminat
16	Suprimir element	EXCEPTION: L'estructura està buida

Descripció de Templates en C++

Definició de Templates

Un "**class template**" en C++ és una característica que permet definir una plantilla per a la creació de **classes genèriques**.

- Aquestes plantilles poden tenir **un o més paràmetres de tipus** que actuen com a marcadors de posició per a tipus específics que es determinaran més endavant quan s'utilitzi la plantilla.

Exemple Template

```
template <class T>
class LaMevaClasse {
private:
    T dada;
public:
    LaMevaClasse(T valor) : dada(valor) {}
    T obtenirDada() const { return dada; }
    void establirDada(T novaDada) { dada = novaDada; }
};
```

En aquest exemple, LaMevaClasse és un class template amb un únic paràmetre de tipus T. Això significa que podem crear instàncies de LaMevaClasse amb diferents tipus de dades. Hem posat nom T, però es pot dir com vulgueu el tipus.

Ús del Class Template

```
int main ()
{
    LaMevaClasse<int> objecteInt(10); // Crear una instància amb tipus int
    LaMevaClasse<double> objecteDouble(3.14); // Crear una instància amb tipus double
    // Utilitzar mètodes de les instàncies
    int valorInt = objecteInt.obtenirDada();
    double valorDouble = objecteDouble.obtenirDada();
    objecteInt.establirDada(20);
    objecteDouble.establirDada(6.28);
}
```

Aquest és només un exemple bàsic. Els class templates en C++ poden ser molt més complexos i poden tenir múltiples paràmetres de tipus, a més ofereixen una àmplia gamma de funcionalitats i tècniques avançades de programació genèrica

Una altre exemple Class Template

```
template<class ItemType>
class GList
{
 public:
    bool IsEmpty() const;
    bool IsFull() const;
    int  Length() const;
    void Insert( /* in */ ItemType item );
    void Delete( /* in */ ItemType item );
    bool IsPresent( /* in */ ItemType item ) const;
    void SelSort();
    void Print() const;
    GList();                    // Constructor
 private:
    int      length;
    ItemType data[MAX_LENGTH];
};
```

Definició del tipus genèric

Template parameter

Instanciant un Class Template

Es poden crear llistes de diferents tipus:

```
// Client code

GList<int> list1;
GList<float> list2;
GList<string> list3;

list1.Insert(356);
list2.Insert(84.375);
list3.Insert("Muffler bolt");
```

template argument

El compilador genera tres llistes diferents

```
GList_int list1;
GList_float list2;
GList_string list3;
```

Amb *Templates* cal linkar el codi de manera diferent. Hi ha vàries maneres:

La més simple és incorporar en el .h, la part d'especificació i la d'implementació (és a dir, no feu el .cpp, cal incloure el codi del .cpp al .h)

Exemple de Template Complexe.h

```
#ifndef COMPLEXE_H
#define COMPLEXE_H

template <class T>
class Complexe{
    private:
        T v_real;
        T v_imaginari;
    public:
        Complexe<T>(T r=0, T i=0);
        T real() const;
        T imaginari() const;
};
template <class T> Complexe<T>::Complexe(T r , T i) : v_real(r), v_imaginari(i){}
template <class T> T Complexe<T>::real() const { return v_real;}
template <class T> T Complexe<T>::imaginari() const {  return v_imaginari; }
#endif
```

Sempre posarem template <class Nom>, encara que a C++ també es pugui declarar com a **template <typename Nom>**

Ús del Template

```
#include "Complexe.h”
using namespace std;

int main() {
    Complexe<float> c(2.5, 3.2);
    cout << "Part real " << c.real() << endl;
    cout << "Part imaginaria " << c.imaginari() << endl;

    Complexe<int> c2(2, 3);
    cout << "Part real " << c2.real() << endl;
    cout << "Part imaginaria " << c2.imaginari() << endl;

    Complexe<int> c3(1, 2);
    cout << "Part real " << c3.real() << endl;
    cout << "Part imaginaria " << c3.imaginari() << endl;

    Complexe<double> c4(0.0,0.0);
    cout << "Part real " << c4.real() << endl;
    cout << "Part imaginaria " << c4.imaginari() << endl;
    return 0;
}
```

Links interessants

- Al campus virtual trobareu:
  - un document anomenat ***Exemple Templates Complex*** que implementa l'exemple de Complex fet amb *class template*
  - Obriu el projecte a Visual Studio Code i testejeu el codi
  - **Fixeu-vos que el .h té tot el codi, l'especificació i la implementació**

- Links externs per tenir més informació:
  - http://www.cplusplus.com/doc/oldtutorial/templates/
  - https://youtu.be/scUUp4-7Cn4

Laboratori 4. Pràctica 2

Estructura de Dades

Grau en Enginyeria Informàtica

Facultat de Matemàtiques i Informàtica,

Universitat de Barcelona